bank.ly Test Cases

All tests use synthetic banking data. Transfers and subscription cancellations are simulated.

The recorded test run completed with no failures among the executed tests. The suite covers agent behavior, backend workflows, financial analysis, security policies, subscription management and shared contracts.

1. How to run

Run from the repository root with the project virtual environment active.

Purpose	Command	Expected
Whole suite	`PYTHONPATH=backend python -m pytest -q --ignore=agent/tests/test_providers.py`	796 passed, 22 skipped
One area	`PYTHONPATH=backend python -m pytest agent -q` (or `backend`, `security`, `finance`, `subscriptions`, `tests`)	all passed
List what is collected	`PYTHONPATH=backend python -m pytest --collect-only -q`	every test id
Subscription cross-check	`python - < p5_check.py`	`TOTAL: 27 OK, 0 MISMATCH`
Subscription CLI	`python -m subscriptions.cli fixtures/transactions.json`	StreamBox and CloudMusic listed

Recorded result: **796 passed, 22 skipped, 0 failed** (818 collected).

Environment note: `agent/tests/test_providers.py` imports the `google` package, which the default environment does not install, so it is excluded above. It tests the live model provider, which the demo does not need (the deterministic mock provider is used). To run it, install the provider dependency first; the install instructions should say so.

2. Test inventory by area

Area	Tests	Files	What it covers
Agent, P1 (`agent/tests`)	352	7	Conversation flow and state, turn handling, request wording, service behaviour, tool-result handling, CLI
Backend, P2 (`backend/tests`)	260	25	Approvals and cancellation of pending actions, action execution, daily transfer limits, MFA, safety lock, security bridge and concurrency, migrations, agent continuation, metadata transport, integration with the agent, finance and subscriptions
Security policy, P4 (`security/tests`)	84	1	Permission-tier policy decisions
Finance, P3 (`finance/tests`)	46	2	Analytics, categories and anomaly detection
Subscriptions, P5 (`subscriptions/tests`)	62	5	Detection, stored-record mapping, cancellation decision, walkthrough, CLI
Shared contracts (`tests`)	14	1	Data models and enums shared by all modules
Total	**818**	**41**	796 passed, 22 skipped

Skipped tests (22): all 22 are PostgreSQL-only tests. They run only when the environment variable `MIGRATION_TEST_POSTGRES_ADMIN_URL` points to a disposable PostgreSQL server, which was not available on the machine used (no Docker). Breakdown: `test_migrations.py` 9, `test_security_concurrency.py` 6, `test_approval_cancellation.py` 5, `test_agent_continuation.py` 1, `test_safety_lock.py` 1. They cover migrations on PostgreSQL and concurrent-request (race) behaviour; the same logic is covered on SQLite by the other tests, but the PostgreSQL race behaviour has not been verified on this machine.

3. Test files (per file)

File	Tests	What it covers
`agent/tests/test_service.py`	94	Agent service behaviour*
`agent/tests/test_conversation_acceptance.py`	71	Three-turn acceptance check through the caller helper; no live API calls
`agent/tests/test_request_wording.py`	61	Understanding of request wording*
`agent/tests/test_tool_results.py`	50	Handling of tool results*
`agent/tests/test_conversation_turns.py`	45	Turn-by-turn conversation*
`agent/tests/test_conversation_state.py`	20	Conversation state*
`agent/tests/test_cli.py`	11	Agent command line*
`agent/tests/test_providers.py`	not run	Model providers; needs the `google` package (see section 1)
`backend/tests/test_approval_cancellation.py`	30	Revoke a pending ticket without executing or reversing its banking operation
`backend/tests/test_security_bridge.py`	27	P4 security bridge and application-path enforcement
`backend/tests/test_metadata_transport.py`	20	P2 HTTP metadata is isolated from P1 arguments, digests and approval inputs
`backend/tests/test_migrations.py`	19	Upgrade real disposable schemas; never use the configured application database
`backend/tests/test_agent_continuation.py`	18	Trusted approval results re-enter P1 without executing dependent proposals
`backend/tests/test_mfa.py`	18	P3 OTP/MFA challenge lifecycle through the HTTP endpoints with P2's real executor
`backend/tests/test_p5_integration.py`	16	P5's real functions through HTTP and persisted cancellation
`backend/tests/test_security_integration.py`	16	HTTP evidence that P2 invokes P4 with canonical proposals and current context
`backend/tests/test_daily_transfers.py`	14	Shanghai day boundaries through real P1/P4 and the guarded approval routes
`backend/tests/test_security_concurrency.py`	12	Safety regressions for concurrent Yellow confirmations
`backend/tests/test_p1_integration.py`	11	Agent inside the backend*
`backend/tests/test_safety_lock.py`	11	Persistent, owner-wide repeated-failure protection using P4's real lock policy
`backend/tests/test_finance_signoff.py`	7	P3 sign-off: expected spending outputs for the data the app actually serves
`backend/tests/test_milestone1.py`	7	Milestone 1 flow*

File	Tests	What it covers
`backend/tests/test_policy.py`	7	Policy inside the backend*
`backend/tests/test_security_details.py`	6	Real default policy selection and atomic, exactly-once escalation auditing
`backend/tests/test_action_execution.py`	5	Shared transaction and replay guarantees, including Yellow failure handling
`backend/tests/test_approvals.py`	5	Approvals*
`backend/tests/test_subscription_storage.py`	3	Stored subscription records*
`backend/tests/test_finance_adapter.py`, `test_subscriptions.py`	2 each	Adapters*
`backend/tests/test_clarification.py`, `test_cross_scenario.py`, `test_execution.py`, `test_read_loop.py`	1 each	Single scenarios*
`security/tests/test_policy.py`	84	Comprehensive test suite for the security policy module (P4)
`finance/tests/test_analytics.py`	31	Aggregation, validation and edge-case tests for `analyze_spending` (P3)
`finance/tests/test_categories_anomalies.py`	15	Categorization rules, anomaly rules, safety and isolation tests (P3)
`tests/test_contracts.py`	14	Shared data models and enums*

4. Cross-module scenarios checked by hand

These are the end-to-end checks in section 6: cancel a subscription through chat, approval, execution, repeat protection, restart, balance, blocked requests and the planner.

5. Subscription module (P5) test cases

Rules: debits only; duplicates removed; grouped by merchant; 5% price band; cadence gaps weekly 6 to 8, monthly 27 to 33, annual 360 to 370 days; confidence 2 charges 0.40, 3: 0.70, 4: 0.85, 5 or more: 0.90 (minus 0.10 if amounts vary). The detector never reads the transaction note and never claims cancellation is supported. The stored record is the source of truth for status and support. Decision order: `SUBSCRIPTION_NOT_FOUND`, `AMBIGUOUS_SUBSCRIPTION`, `ALREADY_CANCELLED`, `CANCELLATION_NOT_SUPPORTED`, `ELIGIBLE`; confirmation is always required.

5.1 Detector (`test_detector.py`, 21 tests)

ID	Case	Expected result
D1	Empty input	Empty report
D2	Clear monthly charges	One candidate, monthly
D3	Weekly charges	Detected, weekly
D4	Annual charges	Detected, annual
D5	Single charge	Not a subscription
D6	Two charges	Candidate with low confidence (0.40)
D7	More charges	Higher confidence
D8	Irregular dates	Nothing detected
D9	Very different amounts	Nothing detected
D10	Small price wobble	Detected, confidence lowered
D11	One charge at a new price	No subscription invented
D12	Same merchant, two services	Kept separate
D13	Credits	Ignored
D14	Duplicate transactions	Do not inflate evidence
D15	Name case and spacing	Normalised
D16	Shuffled input order	Same output
D17	Instruction injected in a note	Treated as data; output unchanged
D18	Raw support flag	Always false
D19	Shared fixtures	Only StreamBox (39.00) and CloudMusic (19.00)
D20	Cut-off 0.70	Separates "possible" from "likely recurring"
D21	Three charges with wobble	Stay below 0.70

5.2 Mapping and cancellation decision (`test_mapping.py` 24 tests, `test_eligibility.py` 2 tests)

ID	Case	Expected result
M1	Same id and merchant	Matched
M2	Different ids, same merchant name	Matched by name
M3	Name case and spaces differ	Matched
M4	Unknown merchant or empty store	`SUBSCRIPTION_NOT_FOUND`
M5	Same id, different merchant	Not a match
M6	Another user's record	Never matched
M7	Same merchant, different price	Told apart by price
M8	Same merchant, same price	`AMBIGUOUS_SUBSCRIPTION`
M9	Same merchant, price matches none	`SUBSCRIPTION_NOT_FOUND`
M10	Duplicate id records	`AMBIGUOUS_SUBSCRIPTION`
M11	Active and supported	`ELIGIBLE`
M12	Active but unsupported	`CANCELLATION_NOT_SUPPORTED`
M13	Already cancelled	`ALREADY_CANCELLED`
M14	Cancel, then ask again	Refused
M15	Cancelled and unsupported	`ALREADY_CANCELLED` reported first
M16	Detector flag true, stored false	Stored record wins
M17	Another user's record	Not found
M18	Shared fixtures	StreamBox eligible, CloudMusic not
M19	Enrichment	Support true only for a matched, active, supported record
M20	Enrichment, cancelled record or candidate flag	Neither is trusted
M21	Enrichment	Other fields kept; inputs not modified

5.3 Walkthrough (`test_walkthrough.py`, 12 tests) and CLI (`test_cli.py`, 3 tests)

ID	Case	Expected result
W1	List shared fixtures	Both candidates with evidence ids
W2	Listing alone	No cancellation permission
W3	Only the supported, verified name	Eligible
W4	Cancel once, relist, retry	Still listed as evidence; retry `ALREADY_CANCELLED`
W5	Cancel one subscription	Other record unaffected
W6	Another user	Sees no record
W7	Detected but not stored	Stays a candidate
W8	Confidence level	Never decides match or permission
W9	Alice, iQIYI VIP Monthly (two charges of 45.00, 30 days apart)	`sub_iqiyi_vip_monthly`, monthly, 0.40, evidence `tx_demo_003`, `tx_demo_002`
W10	Alice, candidate id differs from stored id	Maps by name to `sub_iQIYI_01`, eligible
W11	Alice's record, another user	Not found
W12	Alice cancels, then repeats	Repeat refused
C1	CLI text report	Lists StreamBox and CloudMusic, not Metro
C2	CLI `--json`	Valid JSON, money as strings
C3	CLI missing file	Exit code 2, "File not found"

`p5_check.py` repeats 27 of these rules as a readable script (13 listing and Alice cases, 8 detector cases, 6 mapping cases); the last line must be `TOTAL: 27 OK, 0 MISMATCH`.

6. Manual runs against the real backend

Run with `python backend/start.py`, a disposable database outside the repository, and the demo header `X-Demo-User-Id: usr_demo_01` (Alice) unless stated. Details: `expected_outputs.md` sections 7 to 9.

ID	Request	Expected result	Observed
A1	`GET /api/v1/subscriptions`	iQIYI candidate, 45.00, monthly, 0.4	Pass
A2	`GET /api/v1/subscriptions/records`	`sub_iQIYI_01`, active	Pass
A3	`GET /api/v1/subscriptions?account_id=acc_khong_ton_tai`	404	Pass
A4	Chat: "Cancel iQIYI VIP Monthly subscription"	`PENDING_CONFIRMATION`, one proposal for the stored id; record still active	Pass
A5	Confirm without a body	422, nothing changed	Pass
A6	Confirm with `confirmation_id` and `action_digest`	`COMPLETED`, one audit id, record `cancelled`	Pass

ID	Request	Expected result	Observed
A7	Repeat the confirmation	Idempotent replay, same audit id	Pass
A8	Ask to cancel again (new conversation)	`REJECTED`, `ALREADY_CANCELLED`, no action	Pass
A9	Restart the app on the same database	Data preserved; record still `cancelled`	Pass
A10	Balance before and after	15250.00 CNY both times	Pass
A11	As `user_001`: "Cancel CloudMusic subscription"	Unavailable, no action	Pass
A12	"Cancel Netflix subscription"	Not found, verified choices offered, no action	Pass
A13	"Cancel my subscription"	Asks for the full name, no action	Pass
A14	"Cancel StreamBox subscription"	`PENDING_CONFIRMATION`, one proposal; both records still active	Pass
A15	"Cancel iQIYI ... then transfer 45 CNY to Bob"	Cancellation proposed and completed once; transfer proposed afterwards	**Open**: the transfer step was not proposed when run (item for P1 and P2)